

Device Manager API

Developers can access data, extend functionality, or integrate with the system based on APIs provided by the Device Manager service.

The API is provided as [REST](#) , so you need to understand the basics of REST before reading this document.

Basic usage

General agreement

As stated in this document, unless otherwise specified:

1. Define interfaces in RESTful style
2. The content of the request and response data is in JSON format, and the encoding method is unified in UTF-8.
3. Placeholders in curly brackets in URLs, such as `{id}` , need to be replaced with real data when used
4. The time parameter in the data may have two formats: one is unix timestamp (`1558902012`) and the other is ISO 8601 string (`2019-09-19T14:07:06Z`)

API requests are always prefixed with `/api` or `/api2` , such as: `/api/devices/{id}` .
The API uses JSON as a way to serialize data.

Example of launching an API request using the cURL command:

Shell

```
1 curl "https://iot.inhandnetworks.com/api/devices"
```

DM platform server address

Area	Address
China	https://iot.inhand.com.cn
Global	https://iot.inhandnetworks.com

When you actually send an API request, you need to modify the request address to the address of the corresponding server. For example, if you request `/api/devices`, you need to request the full URL of the Chinese server as <https://iot.inhandnetworks.com/api/devices>

Interface authentication

Most interfaces in doc require authentication and authorization to allow access. Interfaces that do not require authentication will be described separately in the interface description. If the authentication parameter is missing in the request or the authentication fails, an error message with HTTP status code `401` will be returned:

JSON

```
1 {
2   "error": "invalid_token",
3   "error_code": 21336,
4   "request": "/api/devices"
5 }
```

API requests use the OAuth2 access token to authenticate the caller, and the token is carried through the HTTP authentication header or the `access_token` parameters of the URL; when carried using the HTTP authentication header, the Bearer authentication type is used.

Example of carrying a token in a URL:

Shell

```
1 curl
   "https://iot.inhandnetworks.com/api/devices?access_token=<your_access_token>"
```

Example of carrying a token in an HTTP header:

Shell

```
1 curl -H "Authorization: Bearer <your_access_token>"
   "https://iot.inhandnetworks.com/api/devices"
```

Pagination

Some interfaces may need to read data paged when the number of record entries returned by the interface is too large. Interfaces that support paging can request data for a specified page with the following parameters:

Parameter key	Description

<code>cursor</code>	Start reading from the first few pieces of data, default value: <code>0</code>
<code>limit</code>	Number of entries fetched per page, default: <code>10</code>

Note: The limit parameter may be ignored by the interface, and the application should obtain the actual valid limit value based on the limit parameter in the returned value.

The result returned by the paging request contains the following parameters to identify the paging of the read data:

Parameter key	Description
<code>cursor</code>	This page skips the number of entries, the default value: <code>0</code>
<code>limit</code>	Number of entries on this page, default value: <code>10</code>
<code>total</code>	Total number of entries

Example of reading page 2 data request by 10 items per page:

Shell

```
1 curl "https://iot.inhandnetworks.com/api/devices?cursor=10&limit=10"
```

Return example:

JSON

```
1 {
2   "cursor": 10,
3   "limit": 10,
4   "result": [
5     {
6       "_id": "5d7f00e31aa8410001c860a6"
7     },
8     {
9       "_id": "5d79bed01aa8410001c88141"
10    }
11  ],
12  "total": 12
13 }
```

Information filtering mechanism

Some interfaces support the use of `verbose` parameters to specify the degree of detail of the returned data; the larger the value, the more detailed the information required. The default value is `10`, the minimum value is `1`, and the maximum value is `100`. See the interface description for the level of detail corresponding to specific numbers.

General return format

Normally, API requests return data in JSON format and always have a `result` field that specifies the requested data. When a request error occurs, the data returned will contain the following information:

Parameter key	Description
<code>error</code>	Wrong name, such as <code>invalid_token</code>
<code>error_code</code>	Error code, such as: <code>21336</code>
<code>params</code>	Error specific parameters, often used to represent specific error fields or resources, such as: <code>["fields"]</code>

Error return example:

JSON

```
1 {
2   "error": "invalid_token",
3   "error_code": 21336,
4   "request": "/api/devices"
5 }
```

Normal return example:

JSON

```
1 {
2   "result": {
3     "name": "foo"
4   }
5 }
```

Common error codes:

Error code	Error name	Description
10001	internal_error	Internal system error
10002	Service unavailable	The service is temporarily unavailable during system maintenance
10005	Permission denied	Insufficient authority
10016	Miss required parameter (\$params)	Missing required parameters
10017	Parameter (\$param)'s value invalid	Request parameter value error
10020	not_found	The requested API does not exist
20006	Resource (\$resource_name) does not exist	The requested resource does not exist
20007	Resource (\$resource_name) already exists	The requested resource already exists and is often used when creating a resource
21336	invalid_token	Authentication failed, token parameter missing or token expired

OAuth2

Get access token

HTTP
1 POST /oauth2/access_token

Users should obtain access tokens through this interface before accessing other interfaces that need authorization.

When obtaining the access token for the first time, you need to use the password type authorization method to obtain access_token and refresh_token ; access_token The default is valid within 1 hour. After timeout, you need to use the refresh_token type authorization method to obtain a new access_token . When you request an API on the server, you should store the refresh_token , and you don't need to request a token every time you initiate an API request.

- Reference to the general OAuth2 specification
- Content-Type must be `application/x-www-form-urlencoded`
- After 6 wrong passwords, lock your account for 120 seconds and send an email notification
- Currently only supports the standard oauth2: `client_credentials` , `password` , `refresh_token`
- AccessToken default expiration time `1h` , RefreshToken default expiration time `15d`
- `client_id` and `client_secret` Can be specified by form or by `Basic Authorization Header` .

Request params:

Attribute	Type	Required	Description
<code>username</code>	string	yes	User name, for user email or user authentication phone number
<code>password</code>	string	yes	Password
<code>grant_type</code>	string	yes	<code>Password</code> or <code>refresh_token</code>
<code>client_id</code>	string	yes	OAuth2 client ID, fixed: <code>000017953450251798098136</code>
<code>client_secret</code>	string	yes	OAuth2 client secret, fixed: <code>08E9EC6793345759456CB8BAE52615F3</code>
<code>password_type</code>	int	no	Password type, <code>1</code> : original text <code>\$rawPassword</code> , <code>2</code> : <code>md5(\$rawPassword)</code> , default: <code>2</code>
<code>language</code>	int	no	The language of password email notification is <code>1</code> in English and <code>2</code> in Chinese.

Request token request example:

HTTP

```
1 POST /oauth2/access_token HTTP/1.1
2 Authorization: Basic
  MTc5NTM0NTAyNTMyNTk4Mzc1NTI6QjlCNzQzMEQyRTEzMjE2NEM5MTEzRDQyMzRGQUQxREQ=
3 Content-Type: application/x-www-form-urlencoded; charset=utf-8
4
5 grant_type=password&username=user&password=cee6dce05398779e4aaa5030d899bfc0&password_type=2
```

Example refresh token request:

HTTP

```
1 POST /oauth2/access_token HTTP/1.1
2 Authorization: Basic
  MTc5NTM0NTAyNTMyNTk4Mzc1NTI6QjlCNzQzMEQyRTEzMjE2NEM5MTEzRDQyMzRGQUQxREQ=
3 Content-Type: application/x-www-form-urlencoded; charset=utf-8
4
5 grant_type=refresh_token&refresh_token=a112abfbbc1a2773d8aa24c4f8537922
```

Return example:

JSON

```
1 {
2   "access_token": "d0473f092382354ccb6ea980faa356a8",
3   "expires_in": 3600,
4   "refresh_token": "a112abfbbc1a2773d8aa24c4f8537922",
5   "token_type": "Bearer"
6 }
```

Equipment Resources

Device data structure

JSON example:

JSON

```
1  {
2    "oid": "5d0a6217cf4bba0001fddd0a",
3    "name": "RW6153621477215",
4    "model": "IR600",
5    "modelId": "5cab030d5c00560001b1841b",
6    "address": "address",
7    "location": {
8      "longitude": 104.07467651,
9      "latitude": 30.65794373,
10     "time": "2019-09-18T13:27:21Z"
11   },
12   "businessState": 0,
13   "serialNumber": "RW6153621477215",
14   "owner": "5d0a6217cf4bba0001fddd0e",
15   "createTime": 1566788054,
16   "updateTime": 1568813241,
17   "online": 0,
18   "pubIp": "10.5.17.23",
19   "pubPort": 29053,
20   "syncState": 3,
21   "info": {
22     "platformConfig": "9128618469564fe19d1b9b6f32ffc069",
23     "deviceConfig": "9128618469564fe19d1b9b6f32ffc069",
24     "bootVersion": "0.0.1",
25     "hwVersion": "0.1.1",
26     "swVersion": "v0.2.0-msgpack",
27     "imsi": "460013332262529751",
28     "cid": 58378,
29     "lac": 120,
30     "mnc": 0,
31     "mcc": 460,
32     "rssi": 10,
33     "imei": "351756051523999",
34     "iccid": "1234567890123456789",
35     "lan": {
36       "ip": ["192.168.2.1"],
37       "netmask": ["255.255.255.0"],
38       "mac": "AA:AA:AA:AA:AA"
39     }
40   },
41   "protocol": "mqtt",
42   "logtime": "2019-09-18T13:30:05Z",
43   "_id": "5d6349d6335c8c000178a194"
44 }
```

Field description:

Attribute	Type	Verbose	Description
<code>oid</code>	string	1	User's institution identification
<code>_id</code>	string	1	Device identification
<code>name</code>	string	1	Device name
<code>onLine</code>	number	1	Device online status, <code>1</code> is online, <code>0</code> is offline
<code>model</code>	string	2	Model name
<code>serialNumber</code>	string	3	Device serial number
<code>pubIp</code>	string	4	Device IP address
<code>syncState</code>	number	5	Device configuration synchronization status
<code>address</code>	string	5	Device address
<code>info</code>	object	15	Device Local Information
<code>info.deviceConfig</code>	string	15	The latest configuration version reported by the device
<code>info.platformConfig</code>	string	15	The latest device configuration version obtained by the platform
<code>info.bootVersion</code>	string	15	Device Bootloader Version
<code>info.hwVersion</code>	string	15	Device hardware version
<code>info.swVersion</code>	string	15	Device firmware version
<code>info.imsi</code>	string	15	Device IMSI
<code>info.imei</code>	string	15	Equipment IMEI
<code>info.iccid</code>	string	15	Device ICCID
<code>info.rssi</code>	number	15	Equipment signal strength value in <code>asu</code>
<code>info.mcc</code>	number	15	Equipment Base Station MCC
<code>info.mnc</code>	number	15	Equipment Base Station MNC
<code>info.lac</code>	number	15	Equipment Base Station LAC
<code>info.cid</code>	number	15	Device base station CID

<code>info.lan.ip</code>	array	15	Device LAN port IP address, possibly multiple
<code>info.lan.netmask</code>	array	15	Device LAN port subnet mask, possibly multiple
<code>info.lan.mac</code>	string	15	Device LAN port MAC address
<code>info.wan.ip</code>	string	15	Device WAN port IP address
<code>info.wan.mac</code>	string	15	Device WAN port MAC address
<code>location</code>	object	50	Device location information
<code>location.longitude</code>	number	50	Device location longitude
<code>location.latitude</code>	number	50	Equipment location latitude
<code>location.time</code>	string	50	Device location update time
<code>createTime</code>	number	50	Creation time
<code>updateTime</code>	number	50	Update time
<code>protocol</code>	string	50	Device communication protocol
<code>logtime</code>	string	50	Last time the device was online or offline

The field corresponding to info will have a value only after the device reports the relevant data. Otherwise info may be null or have no relevant fields.

Query device list

HTTP
1 <code>GET /api/devices</code>

Request parameters:

Attribute	Type	Required	Description
<code>cursor</code>	int	no	Support paging
<code>verbose</code>	int	no	The level of detail,
<code>name</code>	string	no	Device name
<code>model</code>	string	no	Model name, transferable value: <code>IR900</code> , <code>IR600</code> , etc

onLine	int	no	Device online status
serial_number	string	no	Device serial number

Verbose = 100, which means to get all the information. For complete information, refer to the instance returned in "Query Device Details" below.

Return example:

JSON

```
1 {
2   "cursor": 0,
3   "limit": 10,
4   "total": 1,
5   "result": [{ "name": "foo", "_id": "5cd67d874dd962859f18e327" }]
6 }
```

Query device details

HTTP

```
1 GET /api/devices/{deviceId}
```

Request parameters:

Attribute	Type	Required	Description
deviceId	string	yes	Device ID
verbose	int	no	Information level of detail, 100 means view all information

Request example:

HTTP

```
1 GET /api/devices/603de22827b5c60001b027fe?verbose=100
```

Return example:

JSON

```
1  {
2    "result": {
3      "oid": "5df0c5bbb66af6000133eb00",
4      "name": "GZ5021234213151",
5      "serialNumber": "GZ5021234213151",
6      "model": "IG502",
7      "modelId": "5f8ec64106bc08f655e7844d",
8      "online": 0,
9      "logtime": "2021-03-02T07:16:51Z",
10     "protocol": "mqtt",
11     "config": {
12       "sync": 2,
13       "timeout": 300000,
14       "ackTimeout": 120000,
15       "ackRetries": 3
16     },
17     "pubIp": "10.5.17.24",
18     "pubPort": 2699,
19     "syncState": 3,
20     "info": {
21       "platformConfig": "d196f627fbf739a76c0981f811078958",
22       "deviceConfig": "e10adc3949ba59abbe56e057f20f883e",
23       "bootVersion": "bootV1.19",
24       "hwVersion": "hwV1.23",
25       "swVersion": "swV1.00",
26       "imsi": "imsi39841882",
27       "cid": 200,
28       "lac": 5,
29       "mnc": 0,
30       "mcc": 460,
31       "rssi": 29,
32       "reconnectReason": "poweroff",
33       "wan": {
34         "net": "wifi",
35         "delay": "200",
36         "ip": "216.12.12.12",
37         "interface": "celluar"
38       },
39       "lan": {
40         "ip": ["192.168.2.1", "192.168.2.2"],
41         "netmask": ["255.255.255.0", "255.255.255.0"],
42         "mac": "00:E0:4C:68:02:DC",
43         "interface": "1231234"
44       },
45       "imei": "imei39942033",
46       "iccid": "iccid7200048",
```

```

47     "ngrokVersion": "1.8",
48     "pn": "IR900-GW-E",
49     "accessMode": 1
50 },
51 "address": "address",
52 "location": {
53     "longitude": 113.55693054,
54     "latitude": 22.19477654,
55     "source": "cellTower",
56     "province": "privince",
57     "time": "2021-03-02T06:58:50Z"
58 },
59 "vendorId": 3,
60 "businessState": 0,
61 "owner": "5df0c5bbb66af6000133eb04",
62 "createTime": 1614668328,
63 "updateTime": 1614668342,
64 "_id": "603de22827b5c60001b027fe"
65 }
66 }

```

Device Flow Interface

On average, the equipment will report the sum of the traffic usage of the local cellular network to the cloud every other hour. The cloud will summarize the traffic data reported by the equipment and count the daily traffic and monthly traffic of the equipment every hour.

Query device monthly traffic

Using this interface, you can query the monthly traffic statistics of the specified device, and the traffic unit is bytes. If you request the traffic used by the device `5e6f222afbcf3e0001e133f4`, `5e23f0e3dd77100001eea00b`, `5e23f0e3dd77100001eea00c` in April 2020, you can send the following request.

HTTP

```
1 POST /api/traffic_month/list?month=202004
```

Request parameters:

Attribute	Type	Required	Description
<code>month</code>	int	yes	Month to query, format: <code>yyyMM</code>
<code>resourceIds</code>	array	yes	Equipment to query <code>_id</code>

Request example:

JSON

```
1 {
2   "resourceIds": [
3     "5e6f222afbcf3e0001e133f4",
4     "5e23f0e3dd77100001eea00b",
5     "5e23f0e3dd77100001eea00c"
6   ]
7 }
```

Return field:

Parameter key	Description
send	Total Uplink Traffic for the Month
receive	Total downlink traffic for the month
max	Maximum daily flow for the month
total	Total traffic sent and received during the month
date	Query the first day of the traffic month, format yyyy-MM-dd
deviceId	Equipment _id

Return example:

JSON

```
1  {
2    "result": [
3      {
4        "send": 0,
5        "receive": 0,
6        "max": 0,
7        "total": 0,
8        "deviceId": "5e23f0e3dd77100001eea00b"
9      },
10     {
11       "send": 308479109,
12       "receive": 307881452,
13       "max": 562532552,
14       "total": 616360561,
15       "date": "2020-04-01",
16       "deviceId": "5e6f222afbcf3e0001e133f4"
17     },
18     {
19       "send": 0,
20       "receive": 0,
21       "max": 0,
22       "total": 0,
23       "deviceId": "5e23f0e3dd77100001eea00c"
24     }
25   ]
26 }
```

Query the daily flow of a single device

Using this interface, you can query the daily traffic summary statistics of the specified device in a given month. You can only query the data of one device for one month at a time, and the traffic unit is bytes.

HTTP

```
1  GET /api/traffic_day?month=202003&device_id=5e6f222afbcf3e0001e133f4
```

Request parameters:

Attribute	Type	Required	Description
<code>month</code>	int	yes	Month to query, format: <code>yyyMM</code>
<code>deviceId</code>	string	yes	Equipment <code>_id</code>

Return field:

Parameter key	Description
<code>send</code>	Total upstream traffic on the day
<code>receive</code>	Total downlink traffic on the day
<code>total</code>	Total traffic received and received on the same day
<code>date</code>	Date, format <code>yyyyMMdd</code>
<code>deviceId</code>	Equipment <code>_id</code>

Return example:

JSON

```
1  {
2    "result": [
3      {
4        "deviceId": "5e6f222afbcf3e0001e133f4",
5        "send": 229336336,
6        "receive": 228246080,
7        "total": 457582416,
8        "date": 20200318,
9        "_id": "5e71a3c9c17f1141a4d4770f"
10     },
11     {
12       "deviceId": "5e6f222afbcf3e0001e133f4",
13       "send": 430825283,
14       "receive": 431120087,
15       "total": 861945370,
16       "date": 20200319,
17       "_id": "5e725a98c17f1141a4ddac3b"
18     }
19     // etc.
20   ]
21 }
```

Query device history signal value

HTTP

```
1 GET /api/devices/{deviceId}/signal
```

Request parameters:

Attribute	Type	Required	Description
<code>begin</code>	string	yes	The start time of the query, using the ISO8601 string with Z representing the time zone. Such as <code>2011-12-03T10:15:30Z</code>
<code>end</code>	string	yes	The end time of the query, using the ISO8601 string of Z for the time zone. Such as <code>2011-12-10T10:15:30Z</code>
<code>deviceId</code>	string	yes	Equipment <code>_id</code>

Return field:

Parameter key	Description
<code>values</code>	A two-dimensional array of signal values, each recorded as a signal value at a point in time, Format: <code>[\$time, \$value]</code> , signal unit: <code>asu</code> , value range: 0~ 31

Return example:

JSON

```
1  {
2    "result": {
3      "columns": ["time", "rssi"],
4      "name": "device_signal",
5      "values": [
6        ["2020-04-01T06:32:03.641Z", 15.0],
7        ["2020-04-01T06:50:49.169Z", 15.0],
8        ["2020-04-01T07:00:45.073Z", 15.0],
9        ["2020-04-01T07:10:26.29Z", 15.0],
10       ["2020-04-01T07:13:18.105Z", 15.0],
11       ["2020-04-01T07:13:18.122Z", 15.0],
12       ["2020-04-01T08:12:14.864Z", 15.0],
13       ["2020-04-01T09:03:42.537Z", 15.0],
14       ["2020-04-01T09:06:52.79Z", 15.0],
15       ["2020-04-01T09:08:24.247Z", 15.0],
16       ["2020-04-01T09:12:54.815Z", 15.0]
17     ]
18   }
19 }
```

Query the clients list connected under the device

HTTP

```
1 GET /api/devices/{deviceId}/gateway/clients
```

Request parameters:

Attribute	Type	Required	Description
deviceId	string	yes	Equipment <code>_id</code>

Return field:

Parameter key	Description
<code>_id</code>	Device ID
<code>oid</code>	Agency ID
<code>count</code>	Number of client sides

clients	Client side list
clients.iface	Interface name
clients.mac	Client side MAC address
clients.ipAddress	Client side IP address
clients.hostname	Client side host name
clients.ttl	Valid period in seconds

Return example:

JSON
<pre> 1 { 2 "result": { 3 "oid": "5ce825cb6823ea00019b5676", 4 "count": 1, 5 "clients": [6 { 7 "iface": "eth0", 8 "mac": "00:01:02:03:04:05", 9 "ipAddress": "192.168.1.163", 10 "hostname": "xxxxxxx", 11 "ttl": 221 12 } 13], 14 "updatedAt": "2021-01-14T06:49:28Z", 15 "_id": "5e6f222afbcf3e0001e133f4" 16 } 17 }</pre>

Bulk query the clients list connected under the device

HTTP
<pre> 1 POST /api/gateway/clients</pre>

Request parameters:

Attribute	Type	Required	Description

deviceIds	string array	yes	Device ID array
-----------	--------------	-----	-----------------

Return field:

Parameter key	Description
_id	Device ID
oid	Agency ID
count	Number of client sides
clients	Client side list
clients.iface	Interface name
clients.mac	Client side MAC address
clients.ipAddress	Client side IP address
clients.hostname	Client side host name
clients.ttl	Valid period in seconds

Request example:

JSON

```
1 {
2   "deviceIds": ["5e6f222afbcf3e0001e133f4", "5e23f0e3dd77100001eea014"]
3 }
```

Return example:

JSON

```
1  {
2    "result": [
3      {
4        "oid": "5ce825cb6823ea00019b5676",
5        "count": 1,
6        "clients": [
7          {
8            "iface": "eth0",
9            "mac": "00:01:02:03:04:05",
10           "ipAddress": "192.168.1.140",
11           "hostname": "xxxxxxx",
12           "ttl": 70
13         }
14       ],
15       "updatedAt": "2021-01-14T07:58:51Z",
16       "_id": "5e6f222afbcf3e0001e133f4"
17     },
18     {
19       "oid": "5ce825cb6823ea00019b5676",
20       "count": 1,
21       "clients": [
22         {
23           "iface": "eth0",
24           "mac": "00:01:02:03:04:05",
25           "ipAddress": "192.168.1.27",
26           "hostname": "xxxxxxx",
27           "ttl": 184
28         }
29       ],
30       "updatedAt": "2021-01-14T07:58:51Z",
31       "_id": "5e23f0e3dd77100001eea014"
32     }
33   ]
34 }
```

Device configuration template

Function description

The device configuration template allows users to create a public template configuration. After the template is created, it can be sent to a certain device or multiple devices. Users can use this function to configure devices in batches, or perform pre-configuration operations before the

devices go online. After the device goes online, it will automatically obtain a list of tasks to be performed and execute these tasks when online.

For the specified devices, the configuration template service will automatically create a "run config apply" task for these devices respectively. The task will be automatically executed when the device is online until the task execution of the device is completed. The completed task may have two states: completed or failed. The failed task can be triggered again manually for re-execution. After triggering the retry, the device task will become “pending” again.

The configuration template is only allowed to define the configuration content when it is created. The configuration content of the configuration template that has been created is not allowed to be modified.

Create configuration template

You must specify the model to create the configuration template, and the configuration template can only be distributed to devices of the same model.

| /drc where drc is an abbreviation for device running config

HTTP	
1	POST /api/drc

Request parameters:

Attribute	Type	Required	Description
content	string	yes	Device configuration content, specific configuration content please refer to the specific model of the device prompt doc.
name	string	yes	Template name
model	string	yes	Model name, such as: IR202 , IR600
contentType	number	no	Configure the format of the content, default value: 0, string text.
desc	string	no	Description content

Request example:

JSON

```
1 {
2   "content": "language english",
3   "name": "example",
4   "model": "IR600",
5   "contentType": 0,
6   "desc": "set language to english"
7 }
```

Return field:

Parameter key	Description
content	Device configuration content, specific configuration content please refer to the specific model of the device prompt doc.
name	Template name
model	Model name, such as: IR202 , IR600
contentType	Configure the format of the content, default value: 0, string text.
desc	Description content
oid	Organization identity
create	Creation time, timestamp in ISO 8601 string format

Return example:

JSON

```
1 {
2   "result": {
3     "oid": "61cc14d5a84310000142b11e",
4     "type": 0,
5     "content": "language english",
6     "contentType": 0,
7     "model": "IR600",
8     "name": "example",
9     "create": "2022-01-17T07:25:08Z",
10    "_id": "61e519d427ce950001263171"
11  }
12 }
```

Query the list of all configuration templates

HTTP

1 GET /api/drc

Request parameters:

Attribute	Type	Required	Description
model	string	no	Model name, such as: IR202 , IR600
name	string	yes	Template name
cursor	int	no	Pagination cursor, query offset
limit	int	no	Limit the number of entries returned per page

Return field:

Parameter key	Description
name	Template name
model	Model name, such as: IR202 , IR600
contentType	Configure the format of the content, default value: 0, string text.
desc	Description content
oid	Organization identity
create	Creation time, timestamp in ISO 8601 string format

Return example:

JSON

```
1  {
2    "result": [
3      {
4        "oid": "5d15761da5ab8600010000a7",
5        "model": "IR600",
6        "name": "example",
7        "create": "2022-01-17T07:56:34Z",
8        "_id": "61e521326f5e5d0001f173d3"
9      }
10   ],
11   "total": 1,
12   "cursor": 0,
13   "limit": 10
14 }
```

Query configuration template details

HTTP

```
1 GET /api/drc/{id}
```

Request parameters:

Attribute	Type	Required	Description
<code>id</code>	string	yes	Template ID, which can be obtained from the template list or the returned result when creating

Return field:

Parameter key	Description
<code>name</code>	Template name
<code>model</code>	Model name, such as: <code>IR202</code> , <code>IR600</code>
<code>contentType</code>	Configure the format of the content, default value: 0, string text.
<code>desc</code>	Description content

oid	Organization identity
create	Creation time, timestamp in ISO 8601 string format
stats	Statistics on the number of devices in each task execution state of the selected device. The corresponding states are: to be executed, being executed, execution failure and execution success.

Return example:

JSON

```
1  {
2    "result": {
3      "oid": "5d15761da5ab8600010000a7",
4      "content": "language english",
5      "contentType": 0,
6      "model": "IR600",
7      "name": "example",
8      "create": "2022-01-17T07:56:34Z",
9      "stats": {
10         "pending": 0,
11         "running": 0,
12         "failed": 0,
13         "completed": 0
14       },
15      "_id": "61e521326f5e5d0001f173d3"
16    }
17  }
```

Delete the specified configuration template

HTTP

```
1  DELETE /api/drc/{id}
```

Request parameters:

Attribute	Type	Required	Description
id	string	yes	Template ID, which can be obtained from the template list or the returned result when creating

Return field:

Parameter key	Description
<code>name</code>	Template name
<code>model</code>	Model name, such as: <code>IR202</code> , <code>IR600</code>
<code>contentType</code>	Configure the format of the content, default value: 0, string text.
<code>desc</code>	Description content
<code>oid</code>	Institutional identity
<code>create</code>	Creation time, timestamp in ISO 8601 string format
<code>stats</code>	Statistics on the number of devices in each task execution state of the selected device. The corresponding states are: to be executed, being executed, execution failure and execution success.

Return example:

JSON

```
1  {
2    "result": {
3      "oid": "5d15761da5ab8600010000a7",
4      "content": "language english",
5      "contentType": 0,
6      "model": "IR600",
7      "name": "example",
8      "create": "2022-01-17T07:56:34Z",
9      "stats": {
10        "pending": 0,
11        "running": 0,
12        "failed": 0,
13        "completed": 0
14      },
15      "_id": "61e521326f5e5d0001f173d3"
16    }
17  }
```

Assign devices to configuration templates

Using this interface, you can add devices to the configuration template. The selected device will automatically create a task to issue the configuration. The configuration template will track the execution status of the task. The configuration template service will regularly query all the tasks that need to be performed by online devices and automatically create tasks for online devices. If a device is added to the template but the device is not online, the corresponding device tasks will not be created immediately, and the tasks will not be automatically created until the device is online.

HTTP

```
1 POST /api/drc/{id}/devices
```

Request parameters:

Attribute	Type	Required	Description
<code>id</code>	string	yes	Template ID, which can be obtained from the template list or the returned result when creating
<code>deviceIds</code>	string array	yes	Device unique ID array, all device IDs assigned to the template are required, and the device IDs can be obtained from the query device list interface

Request example:

JSON

```
1 {
2     "deviceIds": ["5d2d4a9f1aa8410001348355"]
3 }
```

Return field:

Parameter key	Description
<code>result</code>	Adding a successful device ID

Return example:

JSON

```
1 {"result":["61e5225b7bf7f354dd686b65"]}
```

Query all devices selected in the configuration template

Using this interface, you can query all devices that have been specified to perform this configuration.

HTTP

```
1 GET /api/job/{id}/devices
```

Request parameters:

Attribute	Type	Required	Description
<code>id</code>	string	yes	Template ID, which can be obtained from the template list or the returned result when creating
<code>cursor</code>	int	no	Pagination offset
<code>limit</code>	int	no	Limit the number of entries returned per page
<code>status</code>	string	no	Filter the list of devices according to the status of device task execution. Optional values: <code>pending</code> , <code>running</code> , <code>failed</code> , <code>completed</code>

Return field:

Parameter key	Description
<code>id</code>	Entry unique identification
<code>oid</code>	Organization unique ID
<code>did</code>	Device unique ID, you can use the query device details interface to obtain device details
<code>rid</code>	Task unique ID, you can use the query task details interface to obtain task details
<code>status</code>	Task execution status: <code>pending</code> , <code>running</code> , <code>failed</code> , <code>completed</code>

<code>createTime</code>	Creation time, timestamp in ISO 8601 string format
<code>task</code>	Device task information

Return example:

```

JSON
1  {
2    "result": [
3      {
4        "id": "61e5225b7bf7f354dd686b65",
5        "oid": "5d15761da5ab8600010000a7",
6        "did": "5d2d4a9f1aa8410001348355",
7        "rid": "61e521326f5e5d0001f173d3",
8        "status": "pending",
9        "createTime": "2022-01-17T08:01:58Z",
10       "task": {
11         "objectName": "System",
12         "name": "Set Device Running Config",
13         "type": 1,
14         "priority": 20,
15         "data": {
16           "platformType": 0,
17           "deviceType": 0,
18           "deviceContent": "xxx"
19         },
20         "jobRunTime": 0,
21         "numFires": 0,
22         "device": false,
23         "typeName": "SetRunningConfig"
24       }
25     ]
26   },
27   "total": 1,
28   "cursor": 0,
29   "limit": 10
30 }

```

Remove devices that have been assigned to templates

Use this interface to remove devices that have been added to the device template. Please note: Removal does not delete the config running tasks that have been created for the device. For devices that do not create tasks, after removal, the configuration will not be distributed to the corresponding devices. If the devices are online and have been created tasks, the tasks will not be automatically cancelled. The configuration of the device has been issued, and the configuration will not be cleared.

HTTP

```
1 DELETE /api/job/{id}/devices/{deviceId}
```

Request parameters:

Attribute	Type	Required	Description
id	string	yes	Template ID, which can be obtained from the template list or the returned result when creating
deviceId	string	yes	Device unique ID

Return field:

Parameter key	Description
result	Number of devices deleted

Return example:

JSON

```
1 {"result":{"deleted":1}}
```

Re-execute completed equipment tasks

Use this interface to restore the completed or failed tasks to the pending state, and re-execute the task to distributed the corresponding configuration content once when the device is online.

HTTP

```
1 PUT /api/jobs/{id}/devices/{deviceId}/restart
```

Request parameters:

Attribute	Type	Required	Description
<code>id</code>	string	yes	Template ID, which can be obtained from the template list or the returned result when creating
<code>deviceId</code>	string	yes	Device unique ID

Request example:

JSON

```
1 {}
```

Return field:

Parameter key	Description
<code>result</code>	Adding a successful device ID

Return example:

JSON

```
1  {
2    "result": {
3      "id": "61d6c01e4387ca3ed6a9b495",
4      "oid": "61cc14d5a84310000142b11e",
5      "did": "61d3f1c9fc66d10001e663c8",
6      "rid": "61d6c01cfb858500019e9d50",
7      "tid": "61d6c01ec7d07a000119e73c",
8      "task": {
9        "objectName": "System",
10       "name": "Set Device Running Config",
11       "type": 1,
12       "priority": 20,
13       "data": {
14         "platformType": 0,
15         "deviceType": 0,
16         "deviceContent": "xxx"
17       },
18       "jobRunTime": 0,
19       "numFires": 0,
20       "device": false,
21       "typeName": "SetRunningConfig"
22     },
23     "status": "pending",
24     "failedTimes": 0,
25     "updateTime": "2022-01-17T08:55:11Z",
26     "createTime": "2022-01-06T10:10:38Z"
27   }
28 }
```

Message integration service

The DM platform supports message integration services. Users can configure integration services to allow the DM platform to forward some device messages (such as: offline, device information update) to the message server owned by the user.

The server types supported by the integration service are: MQTT, AWS Kinesis, Kafka. When the DM platform receives a message related to the device, it will immediately push notification to the configured message server, so that users can obtain real-time events such as the device's offline, and integrate the online status of the device into their own business systems.

Message description

Message content uses JSON as the data format.

Device on-line and off-line messages

Example device on-line messages:

JSON

```
1  {
2    "topic": "$elms/events/presence/connected/5e23f0e3dd77100001eea005",
3    "clientId": "5e23f0e3dd77100001eea005",
4    "deviceId": "5e23f0e3dd77100001eea005",
5    "oid": "5ce825cb6823ea00019b5676",
6    "qos": 0,
7    "messageId": "5e86f3bed91f570001a050b6",
8    "publishedAt": "2020-04-03T08:28:46Z",
9    "data": {
10     "type": "connected",
11     "timestamp": 1585902526,
12     "port": 6839,
13     "ip": "10.5.17.21"
14   },
15   "event": "connected"
16 }
```

Example device offline messages:

When the device goes live, it receives a message with the topic:

`$elms/events/presence/connected/{deviceId}` . When offline, the topic is

`$elms/events/presence/disconnected/{deviceId}` .

JSON

```
1 {
2   "topic": "$elms/events/presence/disconnected/5e23f0e3dd77100001eea005",
3   "clientId": "5e23f0e3dd77100001eea005",
4   "deviceId": "5e23f0e3dd77100001eea005",
5   "oid": "5ce825cb6823ea00019b5676",
6   "qos": 0,
7   "messageId": "5e86f3bed91f570001a050b6",
8   "publishedAt": "2020-04-03T08:28:46Z",
9   "data": {
10    "type": "connected",
11    "timestamp": 1585902526,
12    "reason": "tcp_closed",
13    "port": 6839,
14    "ip": "10.5.17.21"
15  },
16   "event": "disconnected"
17 }
```

Parameter description:

Parameter key	Description
topic	Message subject sent by the device
deviceId	The device ID that sent the message
messageId	Message ID, unique for each message, the client side can use this field for message deduplicate processing
publishedAt	Time of message sending, RFC 3339 format
event	Event type, device online is <code>connected</code> , offline is <code>disconnected</code>
data.timestamp	The time when the device is online or offline, UNIX timestamp
data.ip	The IP address of the device side connection
data.port	Ports connected on the device side
data.reason	Reason for disconnection

Device information reporting

The router will actively report some basic information of the router itself after the connection is online or restarted.

Payload:

JSON

```
1  {
2    "topic": "v1/5e23f0e3dd77100001eea005/device/info",
3    "clientId": "5e23f0e3dd77100001eea005",
4    "deviceId": "5e23f0e3dd77100001eea005",
5    "oid": "5ce825cb6823ea00019b5676",
6    "qos": 1,
7    "messageId": "5e86f6eed91f570001a05130",
8    "publishedAt": "2020-04-03T08:42:22Z",
9    "data": {
10     "hwVersion": "0.1.1",
11     "bootVersion": "0.0.1",
12     "imsi": "460013332262529751",
13     "imei": "351756051523999",
14     "iccid": "1234567890123456789",
15     "swVersion": "v0.2.0-msgpack",
16     "ngrokVersion": "v1.8",
17     "deviceConfig": "c0658c9c14943ec727c899c345a44124",
18     "lan": {
19       "ip": ["192.168.2.1"],
20       "netmask": ["255.255.255.0"],
21       "mac": "00:E0:4C:68:02:DC"
22     },
23     "wan": {
24       "net": "4G",
25       "interface": "wan0",
26       "mac": "00:18:05:30:50:77",
27       "ip": "10.100.50.21"
28     }
29   },
30   "event": "device.info"
31 }
```

Parameter description:

Parameter key	Description
topic	Message subject sent by the device, v1/{deviceId}/device/info
deviceId	The device ID that sent the message
messageId	Message ID, unique for each message, the client side can use this field for

	message deduplicate processing
<code>publishedAt</code>	Time of message sending, RFC 3339 format
<code>event</code>	Event type, traffic message is <code>device.info</code>
<code>data.swVersion</code>	Device firmware version number
<code>data.imsi</code>	Device IMSI information
<code>data.imei</code>	Device IMEI Information
<code>data.iccid</code>	Device iccid information
<code>data.deviceConfig</code>	Device local operation configuration version number
<code>data.lan.ip</code>	Device LAN port IP address, possibly multiple
<code>data.lan.netmask</code>	Device LAN port subnet mask, possibly multiple
<code>data.lan.mac</code>	Device LAN port MAC address
<code>data.wan.ip</code>	Device WAN port IP address
<code>data.wan.mac</code>	Device WAN port MAC address

Device base station information

The router device will actively report this message when the base station used locally transmits a change or the signal value fluctuates.

Payload:

JSON

```
1 {
2   "topic": "v1/5e6f222afbcf3e0001e133f4/lbs",
3   "clientId": "5e6f222afbcf3e0001e133f4",
4   "deviceId": "5e6f222afbcf3e0001e133f4",
5   "oid": "5ce825cb6823ea00019b5676",
6   "qos": 1,
7   "messageId": "5e86aa5cd91f570001a0486f",
8   "publishedAt": "2020-04-03T03:15:40Z",
9   "data": {
10    "cell": {
11      "mcc": 460,
12      "mnc": 0,
13      "lac": 100,
14      "cid": 230,
15      "rssi": 15
16    },
17    "timestamp": "2020-04-03T03:15:40.495Z"
18  },
19   "event": "lbs"
20 }
```

Parameter description:

Parameter key	Description
topic	Message subject sent by the device, v1/{deviceId}/lbs
deviceId	The device ID that sent the message
messageId	Message ID, unique for each message, the client side can use this field for message deduplicate processing
publishedAt	Time of message sending, RFC 3339 format
event	Event type, traffic message is lbs
data.timestamp	Time of data reporting, RFC 3339 format
data.cell.mcc	MCC information of the base station currently used by the device
data.cell.mnc	The base station MNC information currently used by the device
data.cell.lac	Base station LAC information currently used by the device
data.cell.cid	Cell Id information of the base station currently used by the device

`data.cell.rssi`

The current signal value of the device, in asu

Device flow data

Each device will periodically send this message to report the traffic usage used by the local cellular network, generally once an hour.

Topic: `v1/5e6f222afbcf3e0001e133f4/series`

Payload:

JSON

```
1  {
2    "topic": "v1/5e6f222afbcf3e0001e133f4/series",
3    "clientId": "5e6f222afbcf3e0001e133f4",
4    "deviceId": "5e6f222afbcf3e0001e133f4",
5    "oid": "5ce825cb6823ea00019b5676",
6    "qos": 1,
7    "messageId": "5e86a79dd91f570001a04827",
8    "publishedAt": "2020-04-03T03:03:57Z",
9    "data": [
10     {
11       "id": "300009",
12       "value": 86824,
13       "timestamp": 1585883035,
14       "endTime": 1585883037
15     },
16     {
17       "id": "300011",
18       "value": 93890,
19       "timestamp": 1585883035,
20       "endTime": 1585883037
21     }
22   ],
23   "event": "series"
24 }
```

Parameter description:

Parameter key	Description
<code>topic</code>	Message subject sent by the device, <code>v1/{deviceId}/series</code>
<code>deviceId</code>	The device ID that sent the message

<code>messageId</code>	Message ID, unique for each message, the client side can use this field for message deduplicate processing
<code>publishedAt</code>	Time of message sending, RFC 3339 format
<code>event</code>	Event type, traffic message is <code>series</code>
<code>data[].id</code>	Data ID, <code>300009</code> for downlink traffic, <code>300011</code> for uplink traffic
<code>data[].value</code>	The sum of the upstream and downstream traffic used within the upload time range
<code>data[].timestamp</code>	Start time of data statistics, unix timestamp
<code>data[].endTime</code>	End time of data statistics, unix timestamp

Device grouping

Function description

Device grouping provides the API interface for creating, updating, and deleting device groupings and adding devices to and removing devices from groupings.

Device grouping data structure:

YAML
<pre> 1 _id: 5dee1c459597c96253f7d600 2 oid: 5d3903037ce6fa0001e3c1a5 3 name: 888 4 parent: 5dee1c459597c96253f7d650 5 ancestors: [5dee1c 6 459597c96253f7d651, 5dee1c459597c96253f7d650] 7 depth: 3 8 children: [] 9 createAt: 2019-12-24T05:14:00.833Z 10 updateAt: 2019-12-24T05:14:00.833Z </pre>

Field description

Attribute	Type	Description
<code>_id</code>	string	Device group identification
<code>oid</code>	string	Identification of the organization where the device group is

		located
name	string	Device group name
parent	string	Device grouping parent identity
ancestors	array	Device grouping Ancestral level identification
depth	number	Number of device grouping levels (maximum number of levels is 3)
children	array	Sub-node under device grouping
createAt	date	Creation time
updateAt	date	Update time

Device: Device

YAML

```
1 _id: 5d777d198e383600015bd5aa
2 oid: 5d3903037ce6fa0001e3c1a5
3 name: 123123
4 deviceGroups: [5dee1c459597c96253f7d500]
```

Field description

Attribute	Type	Verbose	Description
deviceGroups	array	50	Device group identification

Note: The fields of the device are not listed in detail here, only the fields related to the device grouping are listed. For the details of the device field, please see the device data structure above

Device grouping related API

1. Create device grouping

HTTP

```
1 POST /api/devicegroups
```

Request example:

JSON

```
1 {
2   "name": "888",
3   "parent": ""
4 }
```

Request parameters:

Attribute	Type	Required	Description
<code>name</code>	string	yes	Name (repeatable)
<code>parent</code>	string	no	Parent device group identification

Return example:

JSON

```
1 {
2   "result": {
3     "_id": "5DDDE1DA5EE87500010000D5",
4     "oid": "5DAD7F355EE8750001000011",
5     "name": "888",
6     "parent": "",
7     "ancestors": [],
8     "depth": 1,
9     "createAt": "2019-12-24T05:14:00.833Z",
10    "updateAt": "2019-12-24T05:14:00.833Z"
11  }
12 }
```

2. Update device grouping

HTTP

```
1 PUT /api/devicegroups/:id
```

Request example:

JSON

```
1  {
2    "name": "8881"
3  }
```

Request parameters:

Attribute	Type	Required	Description
name	string	no	Name (repeatable)

Return example:

JSON

```
1  {
2    "result": {
3      "_id": "5DDDE1DA5EE87500010000D5",
4      "oid": "5DAD7F355EE8750001000011",
5      "name": "8881",
6      "parent": "",
7      "ancestors": [],
8      "depth": 1,
9      "createAt": "2019-12-24T05:14:00.833Z",
10     "updateAt": "2019-12-24T05:14:00.833Z"
11   }
12 }
```

3. Get device grouping

HTTP

```
1  GET /api/devicegroups/:id
```

Return example:

JSON

```
1  {
2    "result": {
3      "_id": "5ddde1da5ee87500010000d5",
4      "oid": "5ddde1da5ee87500010000d5",
5      "name": "8881",
6      "parent": "",
7      "ancestors": [],
8      "depth": 1,
9      "createAt": "2019-12-24T05:14:00.833Z",
10     "updateAt": "2019-12-24T05:14:00.833Z"
11   }
12 }
```

4. Get a list of device groupings

HTTP

```
1 GET /api/devicegroups
```

Request parameters:

Attribute	Type	Required	Description
parent	string	no	If the parameter parent is passed, query all sub-nodes under the parent, otherwise query the first-level DeviceGroup
max_depth	int	no	For the number of recursive query layers, 1 is the default value to find the first layer, 2 will find the sub-node layer, and 3 will find the grandson node layer

Return example:

JSON

```
1  {
2    "result": [
3      {
4        "name": "8881",
5        "depth": 1,
6        "children": [
7          {
8            "name": "super",
9            "parent": "5bab5b3e5b4db619a48b2409",
10           "ancestors": ["5bab5b3e5b4db619a48b2409"],
11           "depth": 1,
12           "children": [
13             {
14               "name": "child",
15               "parent": "5bab66d55b4db620b3fab9fd",
16               "ancestors": [
17                 "5bab5b3e5b4db619a48b2409",
18                 "5bab66d55b4db620b3fab9fd"
19               ],
20               "depth": 2,
21               "children": [
22                 {
23                   "name": "child-child",
24                   "parent": "5bac450520645f00013a6bba",
25                   "ancestors": [
26                     "5bab5b3e5b4db619a48b2409",
27                     "5bab66d55b4db620b3fab9fd",
28                     "5bac450520645f00013a6bba"
29                   ],
30                   "depth": 3,
31                   "_id": "5bac450f20645f00013a6bbb"
32                 }
33               ],
34               "_id": "5bac450520645f00013a6bba"
35             }
36           ],
37           "_id": "5bab66d55b4db620b3fab9fd"
38         }
39       ],
40       "_id": "5bab5b3e5b4db619a48b2409"
41     }
42   ]
43 }
```

5. Delete device grouping

HTTP

```
1 DELETE /api/devicegroups/:id
```

Tip: DeviceGroup with sub-node cannot be deleted

Return example:

JSON

```
1 {
2   "result": {
3     "_id": "5DDDE1DA5EE87500010000D5",
4     "oid": "5DAD7F355EE8750001000011",
5     "name": "8881",
6     "parent": "",
7     "ancestors": [],
8     "depth": 1,
9     "createAt": "2019-12-24T05:14:00.833Z",
10    "updateAt": "2019-12-24T05:14:00.833Z"
11  }
12 }
```

6. Get the list of devices that can be added to the device group

- List displayed in descending order by creation time
- Support fuzzy lookup by name
- Support fuzzy search by serial number
- Support paging, verbose

HTTP

```
1 GET /api/devicegroups/:id/devices/exclusion
```

Request parameters:

Attribute	Type	Required	Description
<code>name</code>	string	no	Device name
<code>serial_number</code>	string	no	Device serial number
<code>cursor</code>	int	no	Pagination identifier, indicating how many entries are skipped, default <code>0</code>

limit	int	no	Pagination ID, indicating the number of entries returned per page, default 10
verbose	int	no	Returns the level of detail of the data, the higher the value, the more complete the returned field, range: 0-100 , default 10

Return example:

JSON

```
1 {
2   "result": [
3     {
4       "name": "RT9151714379098",
5       "_id": "5DD62119D20C290001F8A480"
6     }
7   ],
8   "limit": 10,
9   "cursor": 0,
10  "total": 1
11 }
```

7. Get a list of all devices under the device group

- List displayed in descending order by creation time
- Support fuzzy lookup by name
- Support online/offline search
- Support fuzzy search by serial number
- Support whether to recursively query sub-node
- Support paging, verbose

HTTP

```
1 GET /api/devicegroups/:id/devices
```

Request parameters:

Attribute	Type	Required	Description
name	string	no	Device name

<code>serial_number</code>	string	no	Device serial number
<code>online</code>	string	no	0/1 (0 offline, 1 online)
<code>recursive</code>	boolean	no	True/false (whether to recursively query sub-node, default false)
<code>cursor</code>	int	no	Pagination identifier, indicating how many entries are skipped, default <code>0</code>
<code>limit</code>	int	no	Pagination ID, indicating the number of entries returned per page, default <code>10</code>
<code>verbose</code>	int	no	Returns the level of detail of the data. The higher the value, the more complete the field returned. Range: <code>0-100</code> , default <code>10</code>

Return example:

```

JSON

1  {
2    "result": [
3      {
4        "name": "RT9151714379098",
5        "_id": "5DD62119D20C290001F8A480"
6      }
7    ],
8    "limit": 10,
9    "cursor": 0,
10   "total": 1
11  }

```

8. Add devices in batch to device grouping

```

HTTP

1  POST /api/devicegroups/:id/devices

```

Request example:

JSON

```
1 {  
2   "deviceIds": ["5d3ea0e67ce6fa0001e3c5b2"]  
3 }
```

Attribute	Type	Required	Description
deviceIds	array	no	Device identification

Return example:

JSON

```
1 {  
2   "result": [  
3     {  
4       "name": "RT9151714379098",  
5       "_id": "5DD62119D20C290001F8A480"  
6     }  
7   ]  
8 }
```

9. Batch removal of devices from device groups

HTTP

```
1 POST /api/devicegroups/:id/devices/remove
```

Request example:

JSON

```
1 {  
2   "deviceIds": ["5d3ea0e67ce6fa0001e3c5b2"]  
3 }
```

Attribute	Type	Required	Description
deviceIds	array	no	Device identification

Return example:

JSON

```
1  {
2    "result": [
3      {
4        "name": "RT9151714379098",
5        "_id": "5DD62119D20C290001F8A480"
6      }
7    ]
8  }
```

Remote tunnel

Remote tunneling enables the TCP service proxy of the local device on the router side to the cloud, allowing users to remotely connect the local TCP service provided by the lower device of the router. For example, the telnet function of the remote access router itself is realized; the remote desktop function of the HMI device at the lower end of the remote access router.

Remote tunnel data structure

JSON

```
1  {
2    "_id": "5e4a9a264dd9625d77a064f5",
3    "oid": "5e4b5c444dd962613274d557",
4    "deviceId": "5e4b5c504dd96261378a50a8",
5    "name": "No.1 PLC",
6    "proto": "https | http | tcp",
7    "publicUrl": "http://abcde.ngrok.elms.inhand.design", //
      "tcp://10.5.17.51:1234",
8    "localAddress": "192.168.2.3",
9    "localPort": 80,
10   "connected": true,
11   "stateUpdatedAt": "2020-02-17T21:52:51.080Z",
12   "createdAt": "2020-02-18T11:38:31.080Z",
13   "updatedAt": "2020-02-18T11:38:31.080Z"
14 }
```

Field description

Attribute	Type	Description
<code>oid</code>	string	User's institution identification
<code>_id</code>	string	Tunnel identification
<code>name</code>	string	Tunnel name
<code>deviceId</code>	string	Device identification
<code>therefore</code>	string	Tunnel protocol, support: <code>https</code> , <code>http</code> , <code>tcp</code>
<code>publicUrl</code>	string	Accessible address after successful tunnel connection
<code>localAddress</code>	string	The address of the tunnel home agent, usually an IP address
<code>localPort</code>	int	The port number of the tunnel local agent
<code>connected</code>	bool	Tunnel connection status
<code>stateUpdatedAt</code>	string	Tunnel status update time
<code>createdAt</code>	string	Creation time
<code>updatedAt</code>	string	Update time

Query tunnel list

Query all tunnel information that has been created

HTTP
1 <code>GET /api/touch/tunnels?name&device_id</code>

Query parameters:

Attribute	Type	Required	Description
<code>name</code>	string	no	Tunnel name
<code>device_id</code>	string	no	Device identification

Return example:

JSON

```
1  {
2    "result": [
3      {
4        "_id": "5e4a9a264dd9625d77a064f5",
5        "oid": "5e4b5c444dd962613274d557",
6        "deviceId": "5e4b5c504dd96261378a50a8",
7        "name": "No.1 PLC",
8        "proto": "https | http | tcp",
9        "publicUrl": "http://abcde.ngrok.elms.inhand.design |
10       tcp://10.5.17.51:1234",
11       "localAddress": "192.168.2.3",
12       "localPort": 80,
13       "connected": true,
14       "stateUpdatedAt": "2020-02-17T21:52:51.080Z",
15       "createdAt": "2020-02-18T11:38:31.080Z",
16       "updatedAt": "2020-02-18T11:38:31.080Z"
17     }
18   ]
19 }
```

Create tunnel

HTTP

```
1  POST /api/touch/tunnels
```

Request example:

JSON

```
1  {
2    "name": "foo",
3    "deviceId": "5e4b61024dd9626158f4f3d9",
4    "proto": "tcp",
5    "localAddress": "localhost",
6    "localPort": 22
7  }
```

Return example:

JSON

```
1  {
2    "result": {
3      "_id": "5e4a9a264dd9625d77a064f5",
4      "oid": "5e4b5c444dd962613274d557",
5      "deviceId": "5e4b5c504dd96261378a50a8",
6      "name": "No.1 PLC",
7      "proto": "https | http | tcp",
8      "publicUrl": "http://abcde.ngrok.elms.inhand.design |
tcp://10.5.17.51:1234",
9      "localAddress": "192.168.2.3",
10     "localPort": 80,
11     "connected": "true | false",
12     "stateUpdatedAt": "2020-02-17T21:52:51.080Z",
13     "createdAt": "2020-02-18T11:38:31.080Z",
14     "updatedAt": "2020-02-18T11:38:31.080Z"
15   }
16 }
```

Tunnel Enquiry

HTTP

```
1  GET /api/touch/tunnels/{id}
```

Return example:

JSON

```
1 {
2   "result": {
3     "_id": "5e4a9a264dd9625d77a064f5",
4     "oid": "5e4b5c444dd962613274d557",
5     "deviceId": "5e4b5c504dd96261378a50a8",
6     "name": "No.1 PLC",
7     "proto": "https | http | tcp",
8     "publicUrl": "http://abcde.ngrok.elms.inhand.design |
    tcp://10.5.17.51:1234",
9     "localAddress": "192.168.2.3",
10    "localPort": 80,
11    "connected": "true | false",
12    "stateUpdatedAt": "2020-02-17T21:52:51.080Z",
13    "createdAt": "2020-02-18T11:38:31.080Z",
14    "updatedAt": "2020-02-18T11:38:31.080Z"
15  }
16 }
```

Update tunnel

After modifying the tunnel, the established tunnel connection will be immediately aborted.

HTTP

```
1 PUT /api/touch/tunnels/{id}
```

Request example:

JSON

```
1 {
2   "name": "foo",
3   "proto": "tcp",
4   "localAddress": "localhost",
5   "localPort": 22
6 }
```

Return example:

JSON

```
1  {
2    "result": {
3      "_id": "5e4a9a264dd9625d77a064f5",
4      "oid": "5e4b5c444dd962613274d557",
5      "deviceId": "5e4b5c504dd96261378a50a8",
6      "name": "No.1 PLC",
7      "proto": "https | http | tcp",
8      "publicUrl": "http://abcde.ngrok.elms.inhand.design |
tcp://10.5.17.51:1234",
9      "localAddress": "192.168.2.3",
10     "localPort": 80,
11     "connected": "true | false",
12     "stateUpdatedAt": "2020-02-17T21:52:51.080Z",
13     "createdAt": "2020-02-18T11:38:31.080Z",
14     "updatedAt": "2020-02-18T11:38:31.080Z"
15   }
16 }
```

Delete tunnel

When the tunnel is deleted, the connection is immediately aborted.

HTTP

```
1  DELETE /api/touch/tunnels/{id}
```

Return example:

JSON

```
1  {
2    "result": {
3      "_id": "5e4a9a264dd9625d77a064f5",
4      "oid": "5e4b5c444dd962613274d557",
5      "deviceId": "5e4b5c504dd96261378a50a8",
6      "name": "No.1 PLC",
7      "proto": "https | http | tcp",
8      "publicUrl": "http://abcde.ngrok.elms.inhand.design |
9      tcp://10.5.17.51:1234",
10     "localAddress": "192.168.2.3",
11     "localPort": 80,
12     "connected": "true | false",
13     "stateUpdatedAt": "2020-02-17T21:52:51.080Z",
14     "createdAt": "2020-02-18T11:38:31.080Z",
15     "updatedAt": "2020-02-18T11:38:31.080Z"
16   }
17 }
```

Connection tunnel

After initiating the connection tunnel request, the server will send an instruction to the device to connect the tunnel according to the configuration of the tunnel. After the device completes the connection instruction, the interface will return the latest tunnel status result.

- If the device is not online, an `invalid_state (26002)` error will be returned immediately
- A single organization can connect up to 2 TCP tunnels at the same time. If the limit is exceeded, the interface will return `out_of_limit` error
- When the device connection ngrok connection task fails, the interface will return `device_error` error

HTTP

```
1  PUT /api/touch/tunnels/{id}/connect
```

Return example:

JSON

```
1  {
2    "result": {
3      "_id": "5e4e0bef178931e8f45b0c82",
4      "connected": true,
5      "createdAt": "2020-02-20T04:32:47.231Z",
6      "deviceId": "5e23f0e3dd77100001eea04a",
7      "localAddress": "127.0.0.1",
8      "localPort": 80,
9      "name": "tun412",
10     "oid": "5ce825cb6823ea00019b5676",
11     "proto": "http",
12     "stateUpdatedAt": "2020-02-20T10:46:58.952Z",
13     "updatedAt": "2020-02-20T04:32:47.231Z"
14   }
15 }
```

Disconnect the tunnel connection

The user can request to disconnect the specified tunnel through this interface. The return result will be the closed tunnel state.

HTTP

```
1  PUT /api/touch/tunnels/{id}/disconnect
```

Return example: